Software Engineering - Assignment #04

AI-Assisted Software Evolution, Refactoring and Quality Improvement

Smart Campus Room Availability Checker

Student: **Askarbekova Ingkar** | ID: 24B031655 | KBTU | Date: 8th November 2026

Bolt project: https://bolt.new/p/72131830 |

Task map

Requirement in the assignment	Where in this report
Task #01 - Codebase analysis (context, 3+ risks, plan before editing, one modified AI suggestion)	Section 2
Task #02 - Feature extension (4+ rooms, capacity and equipment filters, demonstrations)	Section 3
Task #03 - Refactoring (separation, less duplication, before/after diagram, 2+ design decisions)	Section 4
Task #04 - Automated testing (10+ tests, AC1-AC5 + F1-F5, boundaries, integration/UI check, actual commands)	Section 5, Appendix B
Task #05 - Quality evaluation (evidence table, how measured, limitation)	Section 6
Task #06 - AI reflection (200-300 words)	Section 7
AI assistant indication and AI-use record	Section 8, Appendix A
Submission checklist and required deliverables	Section 9

0. Submission header

Item	Value
Class time and day	Thursday, 9:00-12:00
AI assistants used	Claude (Anthropic) in the claude.ai chat, model Claude Sonnet 5.5; Bolt.new coding agent, model shown in Bolt: Standard
Test runner	Vitest, command `npx vitest run` (Bolt terminal)
Project link	https://bolt.new/p/72131830
Starting point	Smart Campus checker after Practice #09: shared bookingConfig module, one room, 18 tests passing
Final version	5 fictional rooms, capacity and equipment filters, 12 source modules in 4 layers, 57 new automated tests
Source code	Initial: A4_initial_source.zip (Bolt export before the first change). Final: A4_final_source_code.zip (also in the Bolt project)
Files attached to the email	This report, A4_initial_source.zip, A4_final_source_code.zip (README, tests, scripts and AGENTS.md are inside)
Optional advanced extension	Not used. No new SKILL.md was created. AGENTS.md was reused as in Practice #09.

1. Scope, starting point and acceptance criteria

This assignment extends my Smart Campus prototype (built in Assignment #03, repaired in Practice #08, refactored in Practice #09) from one room to a small catalog of rooms. It is an extension and a refactoring, not a repeat of the Practice #08 debugging exercise. The page only checks availability; it never creates a booking. Out of scope: login, payments, live APIs, database, saved bookings.

1.1 Original criteria that must stay true (AC1-AC6)

ID	Criterion	Protected by
AC1	Valid time: 0 <= start < end <= 1440, and start strictly after now (now = 540, 09:00).	AC1-a ... AC1-f, INT-5, INT-6
AC2	Duration at most 120 minutes (exactly 120 allowed, 121 rejected).	AC2-a, AC2-b
AC3	A blocked room rejects every request.	AC3-a, F3-a, F3-b
AC4	Overlaps are rejected; intervals that only touch at an endpoint are allowed.	AC4-a ... AC4-d, INT-3, INT-4
AC5	Booking data is never mutated.	AC5-a, AC5-b
AC6	Clear feedback on desktop and mobile screens.	Validation tests, UI smoke test, screenshots (visual)

1.2 New requirements

ID	Requirement	Implemented in	Tests
Catalog	At least 4 fictional rooms; each has ID, name, capacity, equipment list, blocked status and its own bookings. Only rooms that match the selected requirements are shown. The check uses the chosen room's bookings.	data/rooms.ts, lib/roomAvailability.ts	Catalog integrity, per-room bookings
F1	Capacity filter: room capacity >= requested attendees.	lib/roomFilter.ts `meetsCapacity`	F1-a ... F1-d
F2	Equipment filter: the room contains ALL selected items.	lib/roomFilter.ts `hasAllEquipment`	F2-a ... F2-d
F3	A blocked room is never reported as available.	lib/checkAvailability.ts, lib/roomAvailability.ts	F3-a, F3-b, AC3-a
F4	Clear empty state when no rooms match.	lib/searchState.ts `getVisibleRooms`, components/RoomList.tsx	F4-a ... F4-c
F5	Changing the selected room refreshes the availability result.	lib/searchState.ts `reduce` (selectRoom)	F5-a ... F5-e

1.3 Baseline before any change

The baseline is the final state of Practice #09. Its tests passed 18 of 18 (Practice #09 report, section 7). Before the first change I ran the tests again, took a screenshot of the UI, and exported the project as A4_initial_source.zip. Bolt version or checkpoint of the baseline: Version 4 at Nov 05 9:57 AM

Baseline fact	Value	Source
Test command and result	`npx vitest run`: Test Files 1 passed (1); Tests 18 passed (18)	Practice #09 report; Figure 1
Source modules	src/App.tsx, src/lib/checkAvailability.ts, src/lib/bookingConfig.ts (+ main.tsx entry)	Practice #09 report, sections 2 and 5
Test file	src/lib/checkAvailability.test.ts (18 tests; checks mostly `status`, not reason text)	Practice #09 report, section 2
Rooms	1 (Room A-101, one booking 10:00-11:00)	Practice #09 report
Known gaps	No tests for parseTime/formatTime; AC6 checked only visually	Practice #09 report, sections 2 and 7

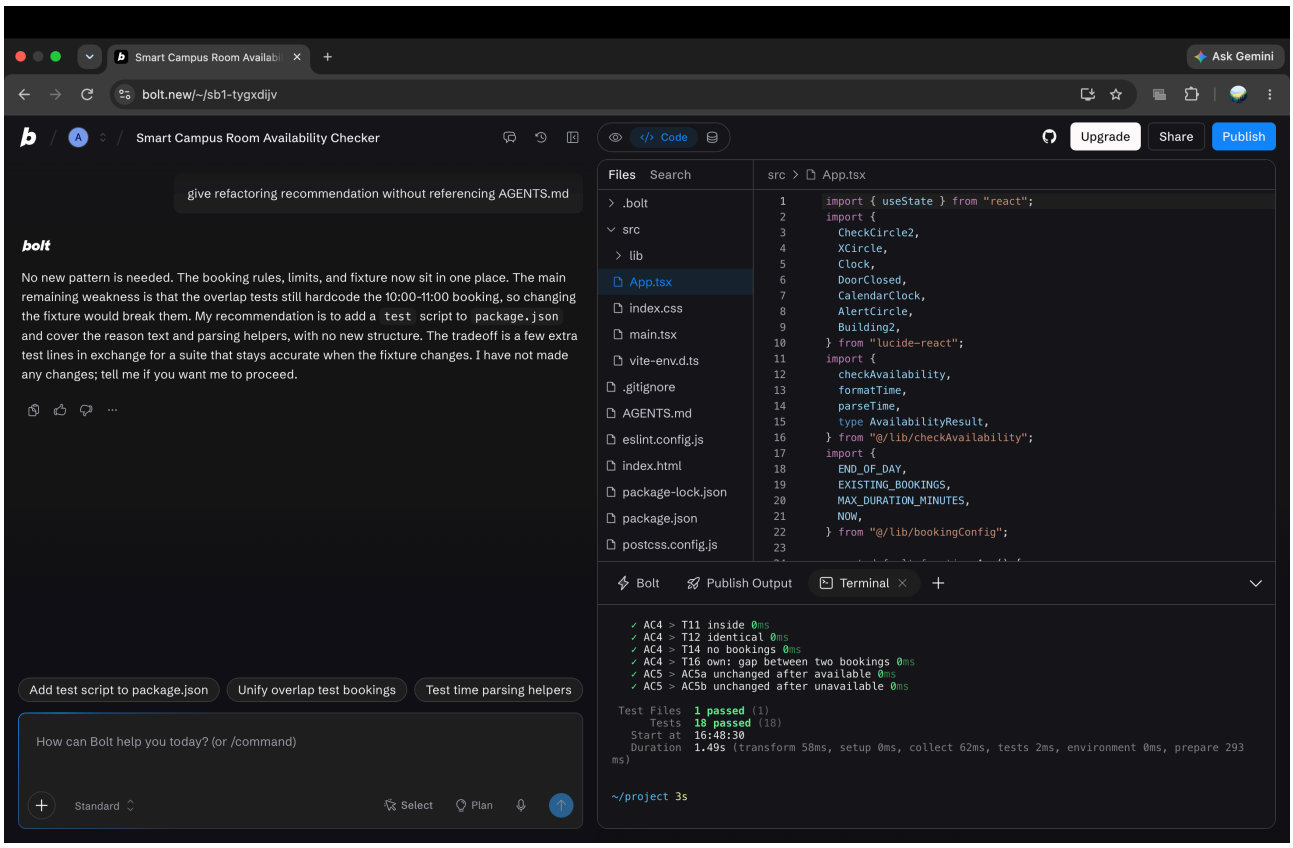


Figure 1. Baseline test run on the initial version (`npx vitest run`, before any change).

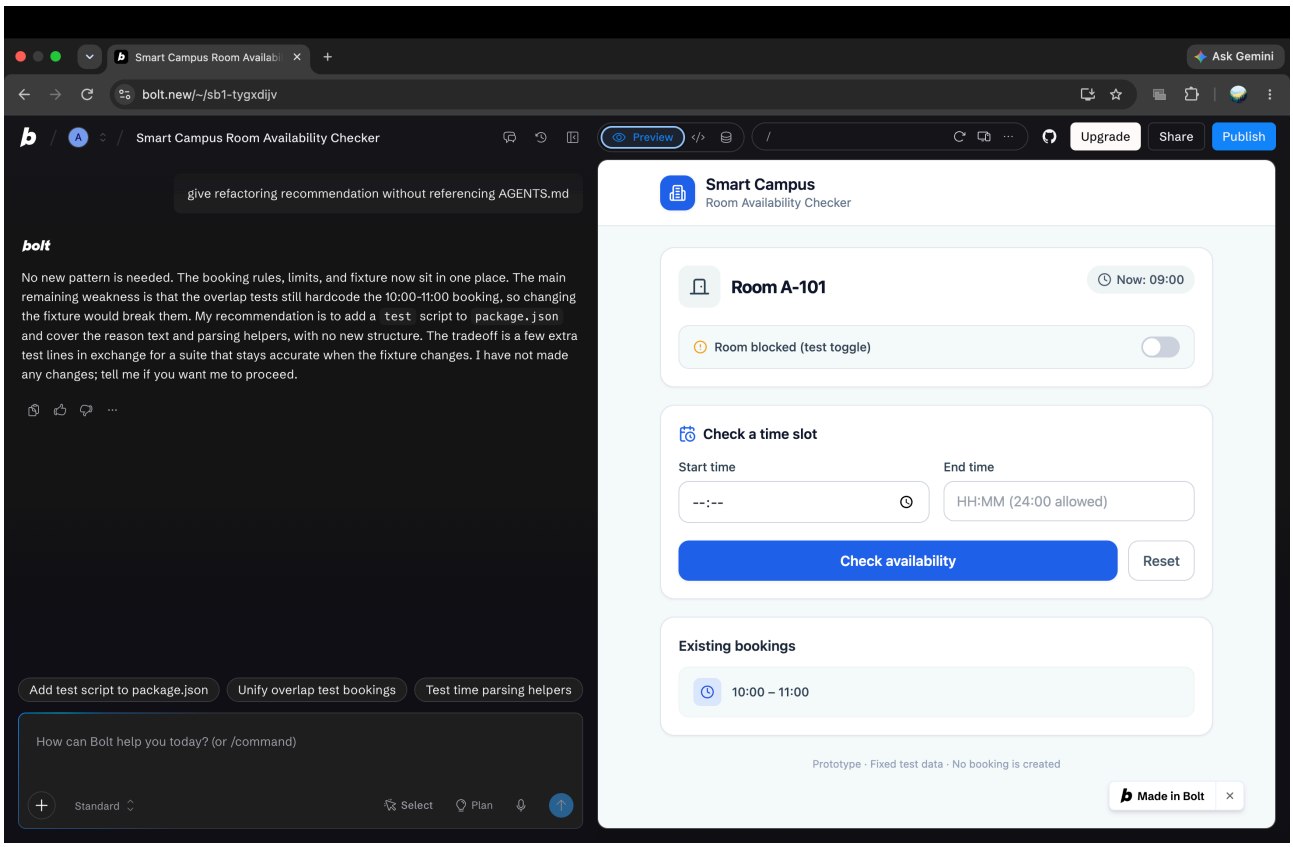


Figure 2. Baseline UI of the initial version at desktop width (one room, Available or Unavailable result visible).

2. Task #01 - Codebase analysis

2.1 Context given to the AI assistant

I gave the assistants three kinds of context: (1) a project file, AGENTS.md, updated for this assignment (Appendix C); (2) a short repository map, the acceptance criteria and the constraints inside the prompt; (3) for Claude, the assignment PDF and my two earlier reports. Bolt does not load AGENTS.md by itself (I learned this in Practice #09), so I first asked it to read the file and summarize it (Prompt B0, Appendix A), then sent Prompt B1.

Prompt B1 (sent to Bolt, read-only):

```
Inspect this Smart Campus room availability project. Do not change any files yet.

Repository map (my notes):
- src/App.tsx: UI
- src/lib/checkAvailability.ts: booking rules (checkAvailability, validateRequest,
  hasOverlap) plus parseTime and formatTime
- src/lib/bookingConfig.ts: MAX_DURATION_MINUTES, END_OF_DAY, NOW, EXISTING_BOOKINGS
- src/lib/checkAvailability.test.ts: 18 Vitest tests

Acceptance criteria that must stay true: AC1 valid time (0 <= start < end <= 1440,
start after now = 540); AC2 max 120 minutes; AC3 blocked room rejected; AC4 overlap
rejected, touching allowed; AC5 booking data never mutated; AC6 clear responsive feedback.

New requirements: a catalog of at least 4 fictional rooms (id, name, capacity,
equipment, blocked, own bookings); F1 capacity >= attendees; F2 ALL selected equipment;
F3 blocked room never available; F4 empty state; F5 room change refreshes the result.

Constraints: no backend, login, payments, API or database; no new dependencies;
do not weaken existing tests.

Tasks: (1) list at least THREE maintainability risks that would make adding the room
catalog harder, each with file and function references; (2) propose a prioritized
improvement plan, smallest safe steps first, BEFORE editing anything; (3) say which
steps carry the highest regression risk for AC1-AC6. Do not write code.
```

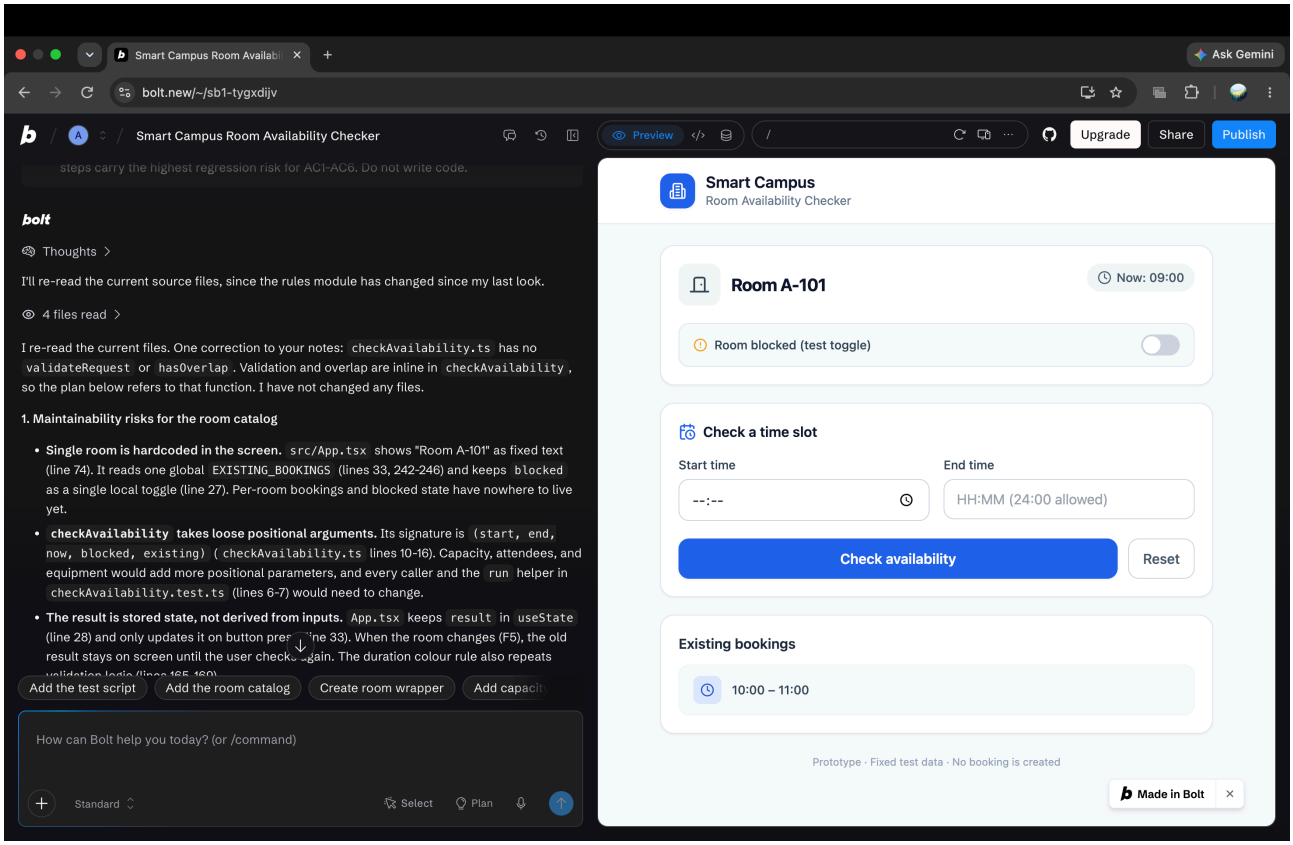


Figure 3. Bolt's inspection response to Prompt B1 (risks, file references, plan). No file was edited.

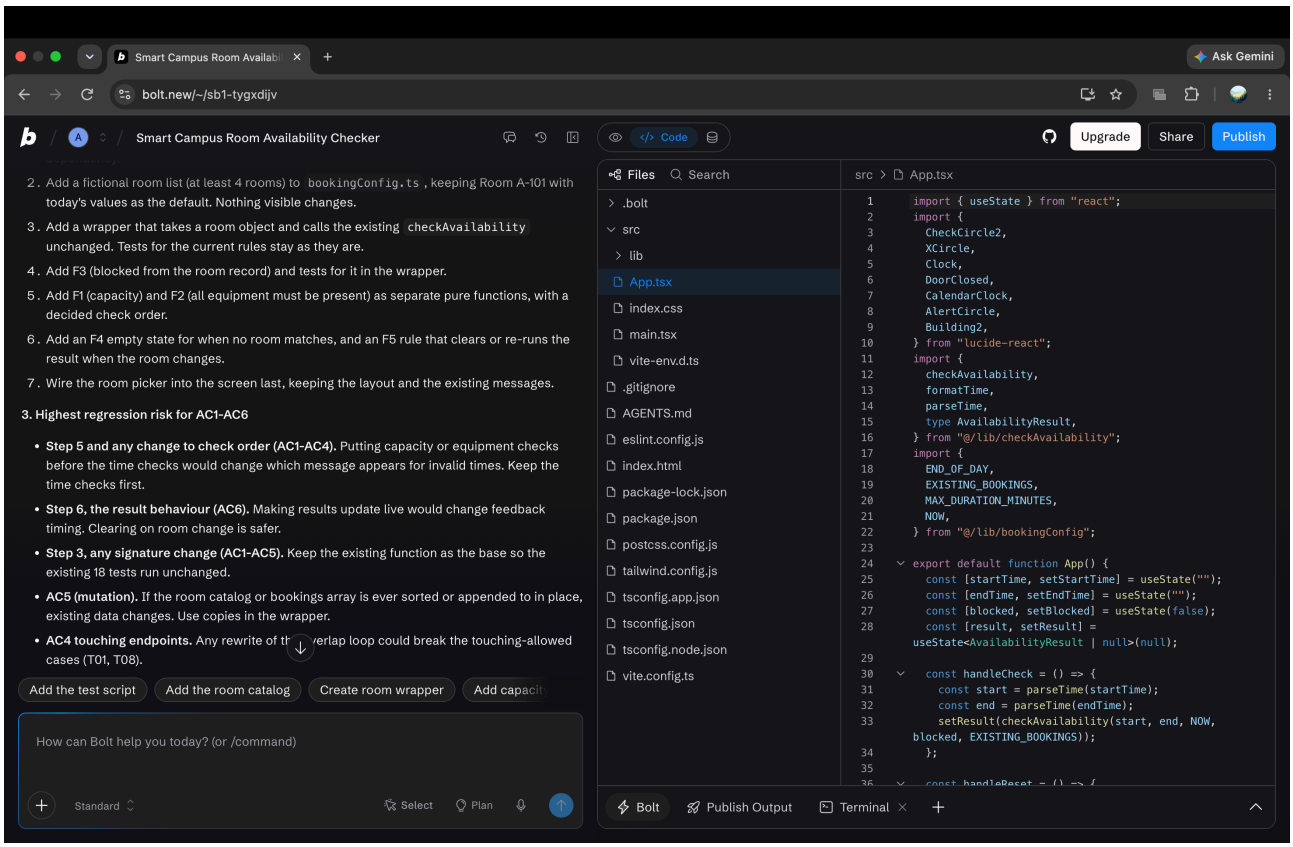


Figure 4. AGENTS.md for Assignment #04 in the Bolt file tree, and Bolt's summary of it (Prompt B0).

2.2 Repository map of the initial version

File	Role	Note
src/App.tsx	UI: time inputs, result display, page state	Single page for ONE room
src/lib/checkAvailability.ts	Booking rules: `validateRequest` (AC1, AC2), `hasOverlap` (AC4), `checkAvailability` (AC3); also `parseTime`, `formatTime`	Pure rules and display helpers in one file
src/lib/bookingConfig.ts	Shared constants and the single-room fixture `EXISTING_BOOKINGS`	Created in Practice #09
src/lib/checkAvailability.test.ts	18 Vitest regression tests	Imports NOW and the fixture from bookingConfig

2.3 Maintainability risks (with file and function references)

I checked each reference by opening the file; Bolt's own inspection (Figure 3) is the second source.

#	Risk	File / function	Why it matters here	Priority
R1	Data for ONE room is global: one booking list and one blocked state.	bookingConfig.ts `EXISTING_BOOKINGS`; App.tsx page state	A catalog would force copying them per room, or the check could read another room's bookings.	High
R2	Rules and display helpers share one file; the UI imports the rules module only to format text.	checkAvailability.ts `checkAvailability` with `parseTime`, `formatTime`	Filtering and UI work would grow this file and mix responsibilities.	Medium

#	Risk	File / function	Why it matters here	Priority
R3	Input handling, calling the rules and rendering result variants live in one component, with no seam for tests.	App.tsx	F4 (empty state) and F5 (room switching) would add more state and conditions to the same file; AC6 is only checked by eye.	High
R4	Weak safety net: tests check `status` only; no tests for time helpers; no UI/state tests.	checkAvailability.test.ts	A rule or message change could pass unnoticed during refactoring.	High

2.4 Prioritized improvement plan (written before editing code)

Step	Change (one at a time)	Why in this order	Check after the step
0	Record the baseline (tests, UI, ZIP). Add AGENTS.md.	No change without a reference point.	Baseline 18/18
1	Data and rules: types, room data, constants, time helpers split from the rules; add AC1-AC5 regression tests and time-helper tests.	Tests first, then the lowest-risk moves; rules keep their signature.	AC tests + original 18
2	Bind one room to the rules (`checkRoom`); add per-room and F3 tests.	Solves R1 before any UI exists.	Per-room tests
3	Add filtering F1 and F2 as pure functions with boundary tests.	Pure code is cheap to test.	Filter tests
4	Move page state into a pure reducer (F4, F5) with integration tests.	Solves R3 testability without new dependencies.	State tests
5	Split the UI into components; one result banner; render smoke test; visual check on desktop and mobile.	UI last: it can only be judged visually.	All tests, tsc, build, screenshots

Highest regression risk: step 1 (the rules are touched) and step 4 (stale results). Both are covered by tests written in the same step.

2.5 AI suggestion I modified

Claude's first version of `parseTime` returned `null` for any time after 24:00. This duplicated the AC1 range rule that `validateRequest` already enforces (end <= 1440). I found it with my own measurement script: it counted 6 implementations for 5 rules, with the 24:00 bound appearing in both `timeUtils.ts` and `checkAvailability.ts`. I modified the code: `parseTime` now only parses, and the range rule lives only in the rules module. A visible side effect is better feedback: "25:00" now gives "Times must be between 00:00 and 24:00." instead of a generic format error. I added test INT-6 for it, and the count went from 6 to 5.

Two other AI outputs were wrong and corrected: the first UI smoke test failed (55 of 56 passed) because React's server renderer inserts comment markers between text and values, so I fixed the test and left the page unchanged; and the first mutation run reported one survivor that I proved to be a quoting bug in the script (applying the mutant by hand made 5 tests fail).

2.6 Suggested AI workflow: how I followed it

Workflow step	What I did	Where
1. Inspect repository and record a baseline	Opened the files, ran the tests, screenshot, exported the initial ZIP	Section 1.3
2. Give AI the project map and explicit requirements	AGENTS.md + Prompt B1	Section 2.1
3. Ask for a small plan, not a rewrite	Prioritized plan before any edit; B1 said "do not write code"	Section 2.4

Workflow step	What I did	Where
4. One change at a time	Five implementation steps	Section 3.3
5. Run tests and review diffs after each change	Test run after each step; Prompt B2 review at the end	Sections 3.3, 4.5
6. Document evidence and limitations	Evidence table, mutation check, limitations	Section 6

3. Task #02 - Feature extension

3.1 Room catalog (fictional data only)

The catalog is data only (`src/data/rooms.ts`). Times are minutes after midnight in the code; the table shows HH:MM. The data was chosen so that every rule has a visible case: an exact-capacity room (B-204 = 12), a blocked room (C-310), a room that matches no equipment combination, and two rooms where the same slot gives different results (A-101 and E-215).

ID	Name	Capacity	Equipment	Blocked	Bookings
A-101	Room A-101	30	Projector, Whiteboard	No	10:00-11:00
B-204	Seminar Room B-204	12	Whiteboard, Video conferencing	No	09:30-10:30, 13:00-14:00
C-310	Computer Lab C-310	24	Computers, Projector, Power outlets	Yes (maintenance)	12:00-13:00
D-001	Lecture Hall D-001	120	Projector, Whiteboard, Video conferencing, Power outlets	No	11:00-12:00, 15:00-16:00
E-215	Study Room E-215	6	Whiteboard	No	12:00-13:00

3.2 How the rules are implemented

The only place that connects a room to the booking rules is `checkRoom`. The original rule function keeps its signature.

```
// src/lib/roomAvailability.ts
export function checkRoom(room: Room, start: number, end: number, now: number) {
  return checkAvailability(start, end, now, room.blocked, room.bookings);
}

// src/lib/roomFilter.ts  (F1 + F2)
export function filterRooms(rooms: readonly Room[], filters: RoomFilters): Room[] {
  return rooms.filter((room) =>
    meetsCapacity(room, filters.attendees) && hasAllEquipment(room, filters.equipment));
}

// src/lib/searchState.ts  (F5: refresh the result when the room changes)
case 'selectRoom': {
  if (!getVisibleRooms(state, ctx).rooms.some((r) => r.id === action.roomId)) return state;
  const next = { ...state, selectedRoomId: action.roomId };
  return { ...next, result: state.result === null ? null : evaluate(next, ctx) };
}
```

Behavior details: attendees empty means no capacity filter; text that is not a whole number of at least 1 shows a validation message; equipment checkboxes are combined with AND; if a filter hides the selected room, the selection and its result are cleared so a stale result is never shown for a hidden room; editing a time clears the old result.

3.3 Implementation log (one change at a time)

After each step I ran the tests that exist at that point and reviewed the diff. The counts below are cumulative results of `npx vitest run <files>`.

Step	Change	New files	Tests passed / total
1	Types, room data, constants, time helpers split from rules, AC1-AC5 regression + time tests	types.ts, data/rooms.ts, lib/bookingConfig.ts, lib/timeUtils.ts, lib/checkAvailability.ts, 2 test files	19 / 19
2	Room bound to rules; per-room and F3 tests	lib/roomAvailability.ts + test	26 / 26
3	Capacity and equipment filtering (F1, F2)	lib/roomFilter.ts + test	37 / 37
4	Pure reducer for page state (F4, F5) + integration tests	lib/searchState.ts + test	54 / 54
5	Components, App composition, UI smoke test; type check and build	4 components, App.tsx, App.render.test.tsx	57 / 57; `tsc -b` exit 0; build OK

The original test file was re-run unchanged after step 1 and after step 5.

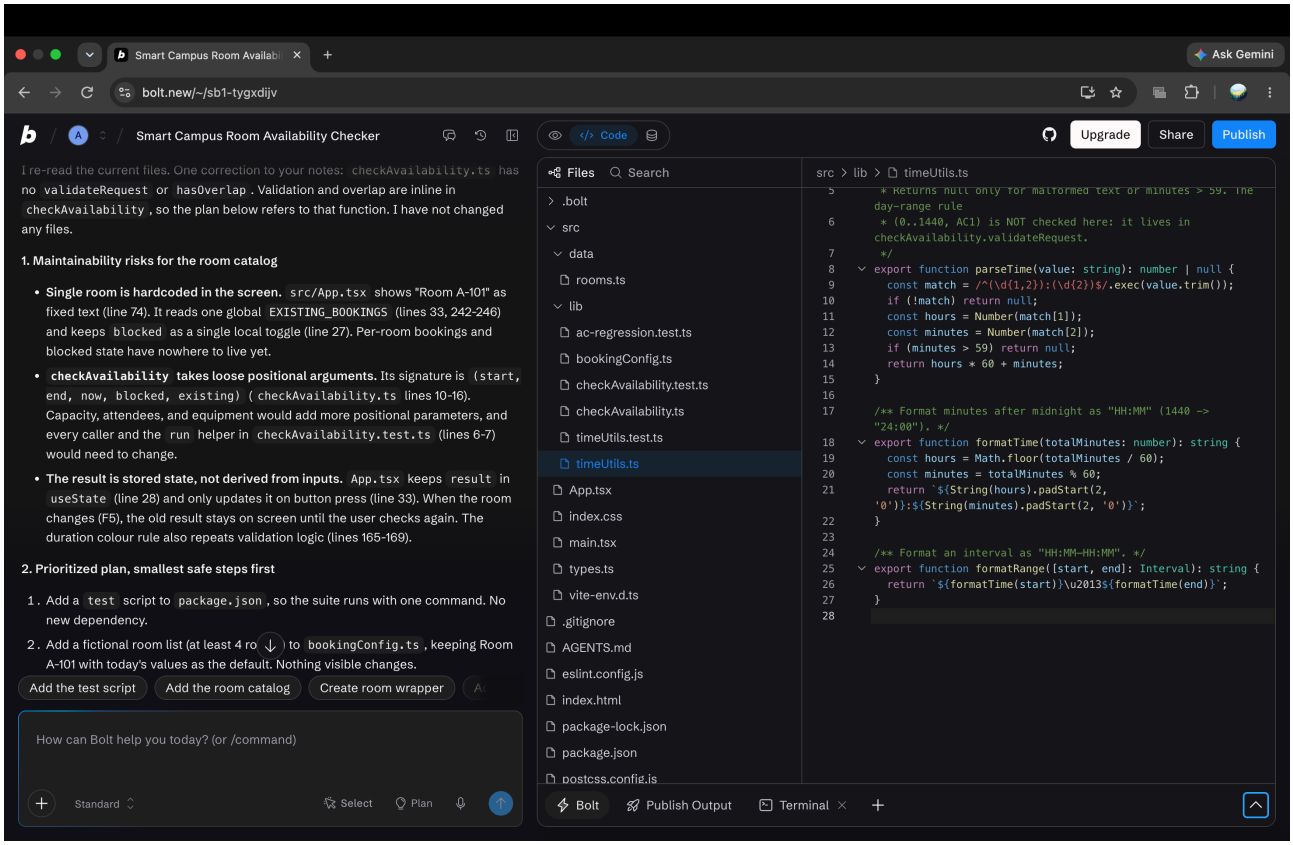


Figure 5. Final project file tree in Bolt (src/data, src/lib, src/components, scripts, AGENTS.md).

3.4 Demonstration: filtering, room switching and availability checks

Demo	What I do (now = 09:00)	Expected result	Automated check
D1	Successful filtering: attendees 12, tick Whiteboard and Video conferencing.	Only B-204 and D-001 are listed.	INT-2, F2-b
D2	Unsuccessful filtering: attendees 500.	Empty state: "No rooms match your requirements."	F4-a, F4-b
D3	Room switching: select A-101, 10:30-11:30, Check; then select E-215.	A-101: Unavailable (overlaps 10:00-11:00). After the switch the result refreshes: E-215 Available.	INT-3, F5-a

Demo	What I do (now = 09:00)	Expected result	Automated check
D4	Blocked room: select C-310, 14:00-15:00, Check.	Unavailable (blocked), although the slot has no booking.	F3-a
D5	Original rules: A-101, 11:00-12:00; then 11:40-13:41.	Available (touching endpoint); Invalid input (121 minutes).	AC4-b, AC2-b
D6	Responsive layout at about 390 px width.	Filters, room cards and result stay readable; no sideways scrolling.	Visual (AC6)

Intentional UI change: the global "Room blocked" test toggle of Assignment #03 was replaced by the per-room blocked flag that this assignment requires (room C-310). AC3 is still tested (AC3-a, F3-a, F3-b).

4. Task #03 - Refactoring

4.1 Responsibilities before and after

Concern	Before (Practice #09 state)	After (final)
Room data	`EXISTING_BOOKINGS` for one room in bookingConfig.ts	`data/rooms.ts`: 5 rooms, data only; `EXISTING_BOOKINGS` is derived from room A-101
Booking / availability rules	checkAvailability.ts (with the time helpers)	`lib/checkAvailability.ts`: rules only (AC1-AC5); `lib/roomAvailability.ts`: binds one room to the rules
Time parsing and formatting	In checkAvailability.ts	`lib/timeUtils.ts`: no booking rules
Filtering logic	None	`lib/roomFilter.ts`: F1, F2, attendees parsing, equipment list
Page state	In App.tsx	`lib/searchState.ts`: pure reducer (F4, F5)
UI presentation	App.tsx (one component)	`App.tsx` (composition) + FilterPanel, RoomList, AvailabilityForm, ResultBanner
Tests	1 file, 18 tests	6 new files, 57 tests (+ the original file, unchanged)

4.2 Before and after diagrams

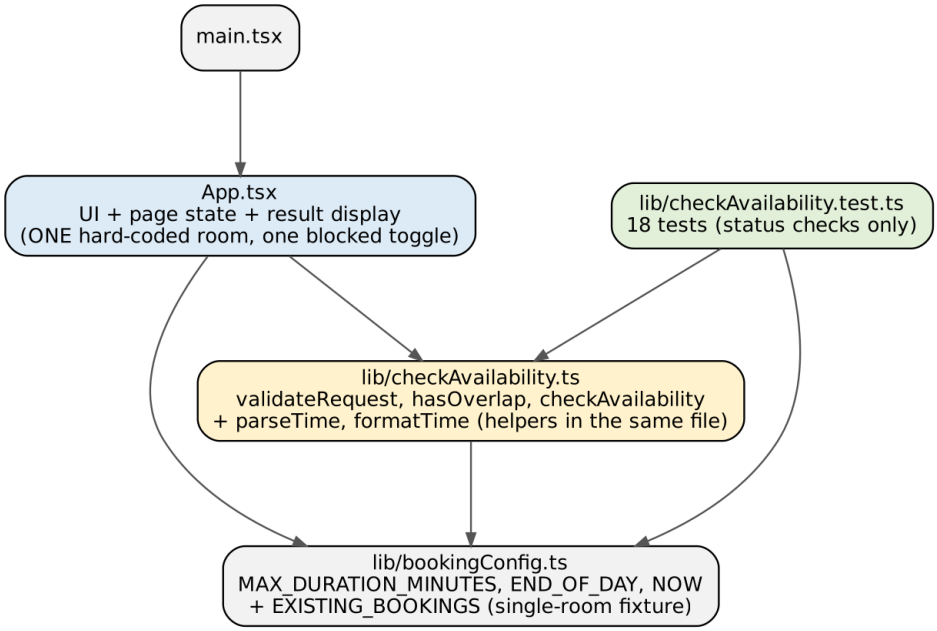


Figure 6. Before: module diagram of the initial version (structure documented in Practice #09, sections 2 and 5).

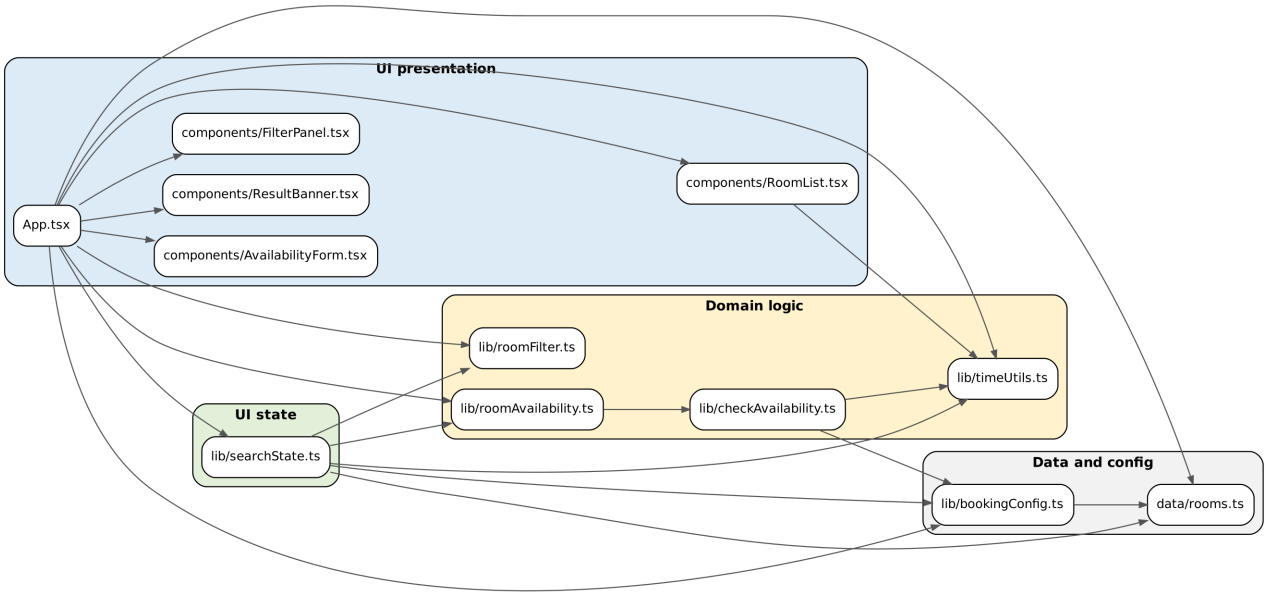


Figure 7. After: module diagram of the final version. Generated from the real import statements of the source files (type-only imports of types.ts left out for readability).

4.3 Design decisions and justification

#	Decision	Why (problem it solves)	Alternative I rejected	Evidence
D1	Keep `checkAvailability(start, end, now, blocked, existing)` unchanged; add `checkRoom(room, ...)` as the only link between a room and the rules.	AC1-AC5 and the original 18 tests stay valid, and "uses the chosen room's bookings" becomes one line that can be tested.	Passing the whole catalog and a room id into the rules (couples the rules to the catalog, breaks the old tests).	Mutant M6 killed (4 tests fail); per-room tests
D2	Separate time parsing/formatting (`timeUtils.ts`) from the rules, and let only the rules module own the day-range rule.	The UI imported the rules file just to format text (R2). The 24:00 bound was checked twice.	Checking the range in both places "for safety" (two rules to keep in sync).	Duplication count 6 to 5; INT-6
D3	Page state as a pure reducer (`searchState.ts`) instead of several `useState` calls in `App.tsx`.	F4 and F5 can be tested without a DOM and without new dependencies (AGENTS.md forbids them); stale-result rules live in one place.	jsdom + Testing Library (new dependencies); keeping the state logic inside the component.	17 state tests; mutant M8 killed
D4	One `ResultBanner` driven by a status-to-style table.	Removes separate result blocks per status; a new status is one table entry.	Three copy-pasted result blocks.	Single rendering location (section 6)
D5	Room catalog is a data-only module and the old single-room fixture is derived from it.	One source of truth for booking data.	A second copy of A-101's bookings in bookingConfig.ts.	Duplication script; catalog tests

Patterns: as in Practice #09 I used only a small module boundary. I did not use Adapter, Strategy, Singleton or Factory because there is one booking shape and one booking policy.

4.4 Duplicated validation and rendering logic that was reduced

- HH:MM parsing and formatting: one implementation each, in `timeUtils.ts`.
- The 120-minute limit and the end-of-day limit: one check each, in `validateRequest`; the constants live in `bookingConfig.ts`.
- The overlap predicate: one implementation, `findOverlap`, which also supplies the conflicting booking for the message.
- Result rendering: one `ResultBanner` component for all three statuses.
- Room data: one catalog; the legacy fixture is derived from it.

How the counts were measured is in section 6.

4.5 Behavior is consistent before and after

The "Before" messages are the ones recorded in my Practice #09 report (section 7); the "After" messages are the actual output of `checkAvailability` on the final code with the same inputs (now = 09:00, A-101 booking 10:00-11:00).

Input	Before (Practice #09)	After (final, actual output)	Same?
Past request 08:00-09:00	Invalid input. "Start time must be strictly after the current time (09:00)."	invalid: "Start time must be strictly after the current time (09:00)."	Yes
121 minutes 11:40-13:41	Invalid input. "Duration cannot exceed 120 minutes."	invalid: "Duration cannot exceed 120 minutes."	Yes
Blocked room 11:40-12:40	Unavailable. "Room is blocked by an administrator."	unavailable: "Room is blocked by an administrator."	Yes
Overlap 10:30-11:30	Unavailable. "Overlaps with existing booking 10:00-11:00."	unavailable: "Overlaps with existing booking 10:00-11:00."	Yes
Touching 11:00-12:00	Available. "No conflicts - the room is free for this time slot."	available: "No conflicts - the room is free for this time slot."	Yes

Evidence that old behavior is preserved: (1) the 15 AC1-AC5 regression tests pass; (2) the original 18-test file is re-run unchanged; (3) the five messages above match; (4) the AC5 tests show the bookings are untouched. After the changes I also asked Bolt to review the code against AGENTS.md (Prompt B2, Appendix A).

5. Task #04 - Automated testing

5.1 Commands and actual results

Results marked "sandbox" were produced by running the commands on the final code in Claude's Linux sandbox (Node 22, Vitest 5.1.0, TypeScript 7.0.2, React 19). I ran the same commands in the Bolt terminal; the screenshots are the evidence for my own environment.

Command	Purpose	Actual result (sandbox)
`npx vitest run`	All automated tests	Test Files 6 passed (6); Tests 57 passed (57); about 0.9 s
`npx tsc -b`	Type check	Exit code 0, no errors
`npm run build`	Type check and production build	27 modules transformed; build completed
`npx vitest run src/lib/checkAvailability.test.ts`	Original Practice #09 test file, unchanged, on the final code	18 passed (18) - expected 18

5.2 Test files

File	Tests	Covers
lib/ac-regression.test.ts	15	AC1-AC5 regression: start == now, past, reversed, end 1440 vs 1441, 120 vs 121, blocked, overlap vs touching, no mutation (also on a frozen array)
lib/roomFilter.test.ts	11	F1, F2, exact vs insufficient capacity, ALL vs ANY equipment, attendees parsing, catalog not mutated
lib/roomAvailability.test.ts	7	Catalog integrity (4+ rooms), per-room bookings, F3 blocked room
lib/searchState.test.ts	17	Integration flow filter, select, check; F4 empty state; F5 room switching; validation feedback; reset

File	Tests	Covers
lib/timeUtils.test.ts	4	HH:MM parsing and formatting, 24:00
App.render.test.tsx	3	UI smoke check: the real `<App />` is rendered to HTML
Total	57	

5.3 Boundary coverage

Boundary	Passing side	Failing side	Tests
Duration 120 vs 121 minutes	700-820 is available	700-821 is invalid	AC2-a, AC2-b
Touching vs overlapping	660-720 and 570-600 are available	630-690, 610-640, 590-670 are unavailable	AC4-a ... AC4-d, INT-3, INT-4
Exact vs insufficient capacity	12 attendees keeps B-204 (capacity 12)	13 attendees drops B-204	F1-a, F1-b
Start equals now vs one minute later	541-600 is available	540-570 is invalid	AC1-a, AC1-c
End of day	end 1440 (23:00-24:00) is available	end 1441 / 25:00 is invalid	AC1-e, INT-5, INT-6
ALL vs ANY equipment	Projector + Video conferencing gives D-001 only	Not "any one of them"	F2-b

5.4 Integration and UI behavior checks

- **Integration (INT-1 ... INT-6):** the same reducer the page uses is driven through the full flow: filter, select a room, type times, check, and switch rooms. It passes through the catalog, the filter, the room binding, the time parser and the rules in one test.
- **UI behavior (F4, F5):** empty state, stale-result clearing and result refresh on room change are tested as state transitions.
- **UI render (App.render.test.tsx):** the real `<App />` is rendered with `react-dom/server` (no new dependency) and checked for all room names, the fixed time, the filter controls, the Blocked badge, and the absence of an empty state or result at start.

5.5 Run history (what failed and what I changed)

Run	Result	Cause	Fix
1	55 passed, 1 failed (56)	The smoke test looked for "Now: 09:00", but React server rendering writes "Now: <!-- -->09:00".	Test strips the markers; page unchanged.
2	56 / 56 passed; `tsc -b` reported TS4104 in the test helper	A frozen (readonly) array was passed to a mutable parameter type.	Helper parameter typed `readonly Interval[]`; tsc exit 0.
3	Duplication script: 6 implementations for 5 rules	The 24:00 bound was checked in `parseTime` and in `validateRequest`.	Removed from `parseTime`; INT-6 added; 57 / 57 passed.

The full list of 57 tests with their requirement and result is in Appendix B.

6. Task #05 - Quality evaluation

6.1 Evidence table: original vs final

I do not invent metrics. Each value is tied to a command or to a file listing, and the last column says how it was obtained.

Measure	Original (Practice #09)	Final (Assignment #04)	How obtained
Automated tests passed / total	18 / 18	57 / 57 in 6 new test files; original 18-test file re-run unchanged: 18 / 18	`npx vitest run`
Implementations of the 5 measured rules (HH:MM parse, HH:MM format, 120-minute check, end-of-day check, overlap predicate); ideal = 5	19/19	5 (0 extra)	`node scripts/duplication-check.mjs src` counts matching lines in non-test source files (patterns in the script); extra matches are duplicates
Duplicated logic locations removed	-	original total minus 5. During the refactoring itself 1 duplicate was found and removed (6 to 5)	Difference of the two script totals; the 6 to 5 change is from my own intermediate run
Result rendering blocks	-	1 (`ResultBanner.tsx`)	Reading the files
Code structure	3 source modules + entry; App.tsx holds UI and page state for one room; rules and time helpers in one file	12 source modules in 4 layers (UI, UI state, domain logic, data/config) + entry; see section 4.1 and the diagrams	File listing, `find src`, import graph
Final size	-	557 lines in 13 non-test modules; 373 lines in 6 test files	`wc -l`
Rooms supported	1	5	src/data/rooms.ts
Type check and build	-	`tsc -b` exit 0; build completed	-
Test strength (mutation spot-check)	not measured	8 / 8 deliberate rule breaks caught	`node scripts/mutation-check.mjs`

If the original total is already 5, the refactoring did not remove duplicates; it prevented the room catalog from creating new ones. I report whichever number the script shows.

6.2 Mutation spot-check (does the suite notice broken rules?)

The script changes one rule at a time, runs the whole suite, restores the file, and reports whether the suite failed ("killed") or still passed ("survived"). Actual output of the final run:

ID	Rule broken	Result
M1	AC2: exactly 120 minutes wrongly rejected	KILLED (1 failed)
M2	AC4: touching intervals treated as overlap	KILLED (5 failed)
M3	AC1: start == now accepted	KILLED (1 failed)
M4	F1: exact capacity rejected	KILLED (3 failed)
M5	F2: ANY instead of ALL equipment	KILLED (17 failed)
M6	Per-room data: another room's bookings used	KILLED (4 failed)
M7	F3: blocked room can be reported available	KILLED (3 failed)
M8	F5: result not refreshed on room change	KILLED (2 failed)

Mutants killed: 8 / 8. The number of failing tests is not a quality measure; it only shows that at least one test noticed the break.

6.3 Remaining limitations

- **UI is not click-tested.** AC6 and the responsive layout are checked visually (desktop and about 390 px). The render test covers only the first render; real clicks are covered at reducer level. A DOM test would need new dependencies, which AGENTS.md forbids.
- **The duplication measure is a line-pattern count.** It can miss a duplicate written in another form and says nothing about design quality. The mutation check covers 8 chosen mutants, not a full mutation score. I did not measure code coverage because it needs an extra package.
- **Scope limits.** The current time is fixed (09:00), the data is fictional and in memory, only one day is modeled, and no booking is created.
- **Legacy coupling.** `EXISTING_BOOKINGS` in bookingConfig.ts exists only so that the original test file keeps working.

7. Task #06 - AI reflection

Context. I gave Claude the assignment PDF, my two earlier reports and the limits from my AGENTS.md: no new dependencies, keep AC1-AC6, keep the signature of checkAvailability. I gave Bolt a repository map, the acceptance criteria and the rule "inspect first, do not edit".

Useful and incorrect suggestions. Useful: the split into data, rules, filtering, state and UI; a pure reducer, which made F4 and F5 testable without new packages; and two small scripts that measure duplication and test strength. Incorrect: parseTime first rejected times after 24:00, which duplicated the AC1 range rule; the first UI smoke test failed because React server rendering inserts comment markers; and the first mutation run reported a survivor because of a quoting bug in the script, not because of a weak test.

What I verified. I opened the files that Bolt and Claude named, re-ran every command in the Bolt terminal, compared the five before and after messages, checked desktop and 390 px screenshots, and applied the surviving mutant by hand before believing the result.

Tests and decisions. The duplication count (6 to 5) made me move the day-range rule out of parseTime. The failing smoke test made me fix the test, not the page. The mutation check showed that all eight deliberate rule breaks are caught, including 120 versus 121 minutes and touching versus overlapping.

Next. I would add click-through tests with Testing Library if dependencies were allowed, check the layout at more widths, and replace the fixed 09:00 with an injectable clock.

(250 words)

8. AI assistant indication and AI-use record

AI assistants used for this assignment: Claude (Anthropic), claude.ai chat, Claude Sonnet 5.5; and Bolt.new (its coding agent inside my project). All AI-generated code and text was reviewed and verified by me before use. The code, tests, scripts, diagrams and the draft of this report were produced with Claude; Bolt was used for inspection, review and as the place where I ran and checked everything.

Tool and date	Prompt summary	Output used	Problems found	Changes made	Verification
Claude, claude.ai, 7 Nov 2026	Prepare the mterials for my report from the assignment PDF and my two earlier reports.	Architecture (5 layers), room catalog, rules/time/filter/state modules, components	parseTime duplicated the 24:00 rule (script count 6 vs 5)	Range rule kept only in validateRequest; INT-6 added	`npx vitest run`, duplication script, section 2.5
Claude, 7 Nov 2026	Same conversation: tests for AC1-AC5, F1-F5, boundaries, integration and UI	57 tests in 6 files	Smoke test failed 55/56 (comment markers); TS4104 in a test helper	Test strips markers; helper typed readonly	Runs 1-3 in section 5.5; `tsc -b`

Tool and date	Prompt summary	Output used	Problems found	Changes made	Verification
Claude, 7 Nov 2026	Same conversation: measurement scripts and diagrams	duplication-check, mutation-check, before/after diagrams	Mutation script showed one false survivor (quoting bug)	Fixed the script; applied the mutant by hand first (5 tests failed)	Mutation run 8/8; diagram built from real imports
Claude, 7 Nov 2026	Same conversation: report text, tables, reflection	This report	Fields that only I can know were left blank	I filled them from my own runs	Commands in section 5; checklist in section 9
Bolt.new, 7 Nov 2026	Prompt B0: read AGENTS.md and summarize; Prompt B1: inspect, 3+ risks, plan, no edits	Cross-check of my risk list (Figure 3)	none found	None (read-only prompts)	I opened every file Bolt named
Bolt.new, 8 Nov 2026	Prompt B2: review the final code against AGENTS.md, no edits	Review list	none found	none	Re-ran all tests after any change

9. Submission checklist and package

9.1 Submission checklist

Checklist item	Evidence	Section
Existing availability rules still work	15 AC1-AC5 regression tests pass; original 18-test file re-run; five identical messages; demo D5	4.5, 5
At least four fictional rooms are included	5 rooms (A-101, B-204, C-310, D-001, E-215)	3.1
Capacity and equipment filters work	F1 and F2 tests (8), demos D1 and D2	3.4, 5.2
At least ten executable tests are provided	57 tests, `npx vitest run`	5, Appendix B
Refactoring and evaluation evidence are included	Diagrams, 5 design decisions, evidence table, mutation check	4, 6
AI use and human review are documented	AI-use record, prompts, review decisions, reflection	2.5, 7, 8, Appendix A

9.2 Required deliverables

Deliverable	Where
Source code, final versions	A4_final_source_code.zip; Bolt project link
README with setup, run and test commands	README.md in the final ZIP (Appendix D)
Test files and actual execution results	src/**/*.test.ts(x) in the final ZIP; section 5
Brief analysis/refactoring report with diagrams and quality comparison	Sections 2, 4, 6 (this report)
AI evidence appendix	Appendix A

Appendix A - AI evidence

Selected prompts, outputs, decisions and verification. Claude's outputs are the files in the final ZIP and the commands in section 5.

#	Tool / prompt (summary)	Response	My review decision and verification
1	Claude: prepare the needed materials for report, using the PDF and my previous reports	Complete code, tests, scripts, diagrams and report draft	Accepted after verification: all commands re-run; blanks filled from my own runs; parseTime duplicate fixed (section 2.5)
2	Bolt, Prompt B0: read AGENTS.md and summarize constraints and commands	Bolt read AGENTS.md itself and summarized the constraints	Confirmed that Bolt read the file itself and did not claim automatic loading
3	Bolt, Prompt B1: inspect, 3+ risks, plan, no edits	Bolt listed maintainability risks with file and function references and proposed a stepwise plan	Compared with my risk table R1-R4; checked file and function names by opening the files
4	Bolt, Prompt B2: review final code against AGENTS.md	Bolt reviewed the code against AGENTS.md.	Checked each claim against the code and the test run

Prompt B0 (Bolt, activation):

```
Read the root AGENTS.md before making changes. Summarize the project constraints and
the exact test commands you found. Do not edit anything yet. If you cannot read
AGENTS.md, tell me and I will paste its content. Do not claim that AGENTS.md was
automatically loaded.
```

Prompt B2 (Bolt, review after the changes):

```
Review the current code against AGENTS.md. Do not edit any files.
(1) List every place where booking-rule logic (time parsing, the 120-minute limit,
the 1440 limit, the overlap check) is implemented more than once.
(2) Check that the signature and AC1-AC5 behavior of checkAvailability are unchanged.
(3) Check that an availability check uses only the selected room's bookings and
blocked flag.
(4) List anything that violates AGENTS.md. Give file and function names.
```

Appendix B - Full list of automated tests and results

Source: `npx vitest run --reporter=verbose` on the final code (57 tests, all PASS).

#	File	ID	Test	Requirement	Result
1	searchState.test.ts	INT-1	initially lists the whole catalog with no empty state	Integration	PASS
2	searchState.test.ts	INT-2	successful filtering: 12 attendees + Whiteboard + Video conferencing -> B-204 and D-001	Integration	PASS
3	searchState.test.ts	INT-3	full flow: select A-101, 10:30-11:30 -> unavailable (overlaps 10:00-11:00)	Integration	PASS
4	searchState.test.ts	INT-4	touching slot 11:00-12:00 in A-101 is available	Integration	PASS
5	searchState.test.ts	INT-5	end time 24:00 is accepted through the whole flow	Integration	PASS
6	searchState.test.ts	INT-6	end time 25:00 gets the AC1 range message from the rules module	AC1 (integration)	PASS
7	searchState.test.ts	F4-a	500 attendees matches no room -> empty state	F4	PASS
8	searchState.test.ts	F4-b	contradictory equipment (Computers + Video conferencing) -> empty state	F4	PASS
9	searchState.test.ts	F4-c	invalid attendees text shows a validation error instead of the empty state	F4	PASS
10	searchState.test.ts	F5-a	switching A-101 -> E-215 recomputes the result for the same times (unavailable -> available)	F5	PASS
11	searchState.test.ts	F5-b	switching to the blocked room C-310 gives unavailable	F5	PASS
12	searchState.test.ts	F5-c	switching before any check does not invent a result	F5	PASS

#	File	ID	Test	Requirement	Result
13	searchState.test.ts	F5-d	a filter that hides the selected room clears the selection and its result	F5	PASS
14	searchState.test.ts	F5-e	a room hidden by the filters cannot be selected	F5	PASS
15	searchState.test.ts	-	asks for a room when none is selected	AC6 feedback	PASS
16	searchState.test.ts	-	asks for HH:MM times when a field is empty	AC6 feedback	PASS
17	searchState.test.ts	-	editing a time clears the stale result, and reset restores the initial state	AC6 feedback	PASS
18	roomFilter.test.ts	F1-a	keeps a room whose capacity equals the attendees (exact capacity: 12 -> B-204)	F1	PASS
19	roomFilter.test.ts	F1-b	drops a room with insufficient capacity (13 -> B-204 excluded)	F1	PASS
20	roomFilter.test.ts	F1-c	with 13 attendees returns exactly A-101, C-310 and D-001	F1	PASS
21	roomFilter.test.ts	F1-d	null attendees applies no capacity filter	F1	PASS
22	roomFilter.test.ts	F2-a	one item: Video conferencing -> B-204 and D-001	F2	PASS
23	roomFilter.test.ts	F2-b	two items must BOTH be present: Projector + Video conferencing -> D-001 only	F2	PASS
24	roomFilter.test.ts	F2-c	an item no room has returns an empty list	F2	PASS
25	roomFilter.test.ts	F2-d	capacity and equipment combine: 30 attendees + Projector -> A-101 and D-001	F2	PASS
26	roomFilter.test.ts	-	does not mutate the catalog	F1/F2 helpers	PASS
27	roomFilter.test.ts	-	parseAttendees: empty = no filter; whole numbers >= 1 are valid; others give an error	F1/F2 helpers	PASS
28	roomFilter.test.ts	-	listEquipment returns the distinct sorted equipment names	F1/F2 helpers	PASS
29	ac-regression.test.ts	AC1-a	rejects a request that starts exactly now (540-570)	AC1	PASS
30	ac-regression.test.ts	AC1-b	rejects a past request (480-540)	AC1	PASS
31	ac-regression.test.ts	AC1-c	accepts a request starting one minute after now (541-600)	AC1	PASS
32	ac-regression.test.ts	AC1-d	rejects reversed (700-650) and zero-length (700-700) requests	AC1	PASS
33	ac-regression.test.ts	AC1-e	accepts end = 1440 (23:00-24:00) and rejects end = 1441	AC1	PASS
34	ac-regression.test.ts	AC1-f	rejects a negative start	AC1	PASS
35	ac-regression.test.ts	AC2-a	accepts exactly 120 minutes (700-820)	AC2	PASS
36	ac-regression.test.ts	AC2-b	rejects 121 minutes (700-821)	AC2	PASS
37	ac-regression.test.ts	AC3-a	never reports a blocked room as available	AC3	PASS
38	ac-regression.test.ts	AC4-a	rejects a partial overlap (630-690)	AC4	PASS
39	ac-regression.test.ts	AC4-b	allows a request touching the end of a booking (660-720)	AC4	PASS
40	ac-regression.test.ts	AC4-c	allows a request touching the start of a booking (570-600)	AC4	PASS
41	ac-regression.test.ts	AC4-d	rejects a request inside a booking (610-640) and one enclosing it (590-670)	AC4	PASS
42	ac-regression.test.ts	AC5-a	leaves the bookings array deep-equal after every outcome	AC5	PASS
43	ac-regression.test.ts	AC5-b	works on a frozen array (would throw if the code tried to mutate it)	AC5	PASS
44	roomAvailability.test.ts	-	has at least four rooms with unique ids and all required fields	Catalog (4+ rooms)	PASS
45	roomAvailability.test.ts	-	the same slot (10:30-11:30) is unavailable in A-101 but available in E-215	Per-room data	PASS
46	roomAvailability.test.ts	-	the reason names the booking of the chosen room (B-204: 09:30-10:30)	Per-room data	PASS
47	roomAvailability.test.ts	-	touching intervals stay available in a non-first room (B-204 10:30-11:30)	Per-room data	PASS
48	roomAvailability.test.ts	-	original validation still applies per room (121 minutes -> invalid)	Per-room data	PASS

#	File	ID	Test	Requirement	Result
49	roomAvailability.test.ts	C-310	is unavailable even for a slot with no bookings (14:00-15:00)	F3	PASS
50	roomAvailability.test.ts	C-310	is never available for any 30-minute slot of the day after now	F3	PASS
51	timeUtils.test.ts	-	parses valid HH:MM values	Time helpers	PASS
52	timeUtils.test.ts	-	parses 24:00 as 1440; the day-range rule is enforced by checkAvailability, not here	Time helpers	PASS
53	timeUtils.test.ts	-	rejects malformed text and minutes above 59	Time helpers	PASS
54	timeUtils.test.ts	-	formats minutes back to HH:MM and ranges with an en dash	Time helpers	PASS
55	App.render.test.tsx	-	shows every room of the catalog	UI smoke (AC6)	PASS
56	App.render.test.tsx	-	shows the fixed current time, the filters and the check button	UI smoke (AC6)	PASS
57	App.render.test.tsx	-	shows the blocked badge for the blocked room and no empty state or result at start	UI smoke (AC6)	PASS

Appendix C - AGENTS.md used for this assignment

```
# AGENTS.md

## Project
Smart Campus Room Availability Checker (KBTU Software Engineering, Assignment #04).
User story: As a student, I want to find a room that fits my group and equipment and check whether
it is free, so I can choose a valid booking time.
Scope: a small fictional catalog of at least 4 rooms, fixed current time (now = 540, i.e. 09:00),
availability decision only.

## Structure
- src/data/rooms.ts - room catalog (data only)
- src/lib/bookingConfig.ts - constants (MAX_DURATION_MINUTES, END_OF_DAY, NOW)
- src/lib/checkAvailability.ts - booking rules AC1-AC5 (pure functions)
- src/lib/timeUtils.ts - HH:MM parsing and formatting
- src/lib/roomFilter.ts - capacity and equipment filtering
- src/lib/roomAvailability.ts - one room + the rules
- src/lib/searchState.ts - pure reducer for page state
- src/components/*.tsx and src/App.tsx - UI only
- src/**/*.test.ts(x) - Vitest tests

## Acceptance criteria (must stay true)
- AC1: reject start >= end and past requests (start must be after now); times within 0-1440.
- AC2: reject durations longer than 120 minutes (exactly 120 is allowed).
- AC3: reject a blocked room.
- AC4: reject overlaps; allow bookings that touch only at endpoints (half-open intervals).
- AC5: never mutate booking data.
- AC6: clear, responsive feedback (desktop and mobile).
- F1: room capacity >= requested attendees. F2: room contains ALL selected equipment.
- F3: a blocked room is never reported as available. F4: clear empty state when no rooms match.
- F5: changing the selected room refreshes the availability result.

## Constraints
- No backend, login, payments, live APIs, database or saved bookings. Fictional data only.
- Do not change the signature or the AC1-AC5 logic of checkAvailability(start, end, now, blocked,
existing).
- Do not add dependencies. Do not weaken or delete existing test assertions.
- One change at a time; run the tests after each change.

## Engineering rules
- Prefer pure functions and the smallest change that solves the problem.
- Keep each rule in exactly one place (no duplicated validation or parsing).
- Do not use Singleton, Factory, Strategy, Adapter or class hierarchies without a concrete need.

## Commands
- Tests: npx vitest run
- Type check: npx tsc -b
- Build: npm run build
- Dev preview: npm run dev

## Reporting
```

```
- Report changed files, the reason for each, the exact test command and output, and remaining
limitations.
- Never claim tests passed without running them. If something cannot be run, say so.
```

Appendix D - README (setup, run and test commands)

````
# Smart Campus Room Availability Checker (Assignment #04 final version)

KBTU Software Engineering, Assignment #04 - Askarbekova Ingkar (24B031655).
Fictional data only. No login, payments, live APIs or database. The page only checks availability;
it never creates a booking.

## Features
- Catalog of 5 fictional rooms (ID, name, capacity, equipment, blocked status, own bookings) in
`src/data/rooms.ts`.
- Filters: attendees (capacity >= attendees) and required equipment (room must have ALL selected
items).
- Pick a room, enter start/end time (HH:MM, end may be 24:00) and check availability against THAT
room's bookings.
- Original rules AC1-AC6 are preserved (fixed current time 09:00, max 120 minutes, blocked room,
half-open overlap, no mutation, clear responsive feedback).

## Setup, run, test
```bash
npm install                 # install dependencies
npm run dev                 # start the dev server (Bolt: preview opens automatically)
npx vitest run              # run ALL automated tests (same as: npm test)
npx tsc -b                  # type check
npm run build               # type check + production build
node scripts/duplication-check.mjs src   # count implementations of each rule (duplication measure)
node scripts/mutation-check.mjs          # break 8 rules one by one; the tests must fail each time
```
Expected final result of `npx vitest run` for the new suite: `Test Files 6 passed (6)`, `Tests 57
passed (57)`.
If the original Practice #09 test file `src/lib/checkAvailability.test.ts` is kept, it runs in the
same command.

## Structure
| Path | Responsibility |
|---|---|
| `src/data/rooms.ts` | Room catalog (data only) |
| `src/lib/bookingConfig.ts` | Constants: `MAX_DURATION_MINUTES`, `END_OF_DAY`, `NOW`;
`EXISTING_BOOKINGS` = A-101 bookings |
| `src/lib/checkAvailability.ts` | Booking rules AC1-AC5 (pure): `validateRequest`, `findOverlap`,
`checkAvailability` |
| `src/lib/timeUtils.ts` | `parseTime`, `formatTime`, `formatRange` (no booking rules) |
| `src/lib/roomFilter.ts` | F1/F2 filtering, attendees parsing, equipment list |
| `src/lib/roomAvailability.ts` | `checkRoom(room, ...)`: binds one room's own bookings and blocked
flag to the rules |
| `src/lib/searchState.ts` | Pure reducer for the page state (F4 empty state, F5 refresh on room
change) |
| `src/components/*.tsx`, `src/App.tsx` | UI presentation only |
| `src/**/*.test.ts(x)` | Vitest tests (6 files, 57 tests) |
| `scripts/` | `duplication-check.mjs`, `mutation-check.mjs` |

## Tests
| File | Tests | Covers |
|---|---|---|
| `src/lib/ac-regression.test.ts` | 15 | AC1-AC5 regression, 120 vs 121, touching vs overlapping, no
mutation |
| `src/lib/roomFilter.test.ts` | 11 | F1, F2, exact vs insufficient capacity, attendees parsing |
| `src/lib/roomAvailability.test.ts` | 7 | catalog integrity, per-room bookings, F3 blocked room |
| `src/lib/searchState.test.ts` | 17 | integration flow, F4 empty state, F5 room switching,
validation |
| `src/lib/timeUtils.test.ts` | 4 | HH:MM parsing and formatting |
| `src/App.render.test.tsx` | 3 | UI smoke check: renders the real `<App />` to HTML |

Known limitation: layout and responsiveness (AC6) are checked visually (desktop and ~390 px); there
is no DOM click-through test because no new dependencies are allowed.
````